KCD BEIJING vLLM

从 Device Plugin 到 DRA：GPU 调度范式升级与 HAMi DRA 实践

王纪飞（密瓜智能）、James Deng（第四范式）

Content

01 DRA Overview

02 Practical Experience with HAMi DRA

03 DRA vs Device Plugin

04 Roadmap & Future Work

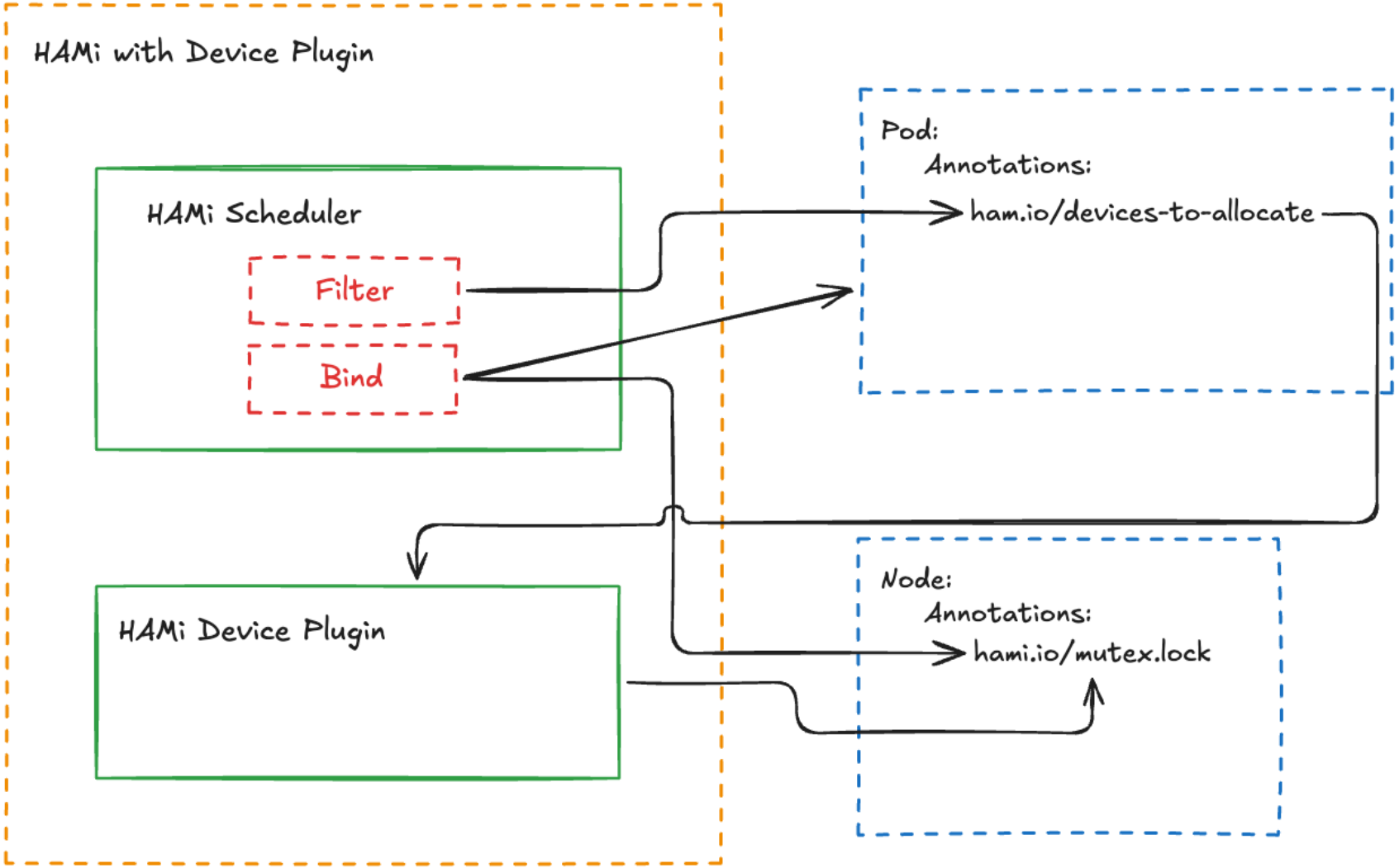
Part 01

DRA Overview

Advantages of DRA

- Multi-Dimensional Resource Modeling Capability
- Comprehensive Device Lifecycle Management
- Fine-Grained Resource Allocation Capability

Node Lock in HAMi



Part 02

Practical Experience with HAMi DRA

DRA Driver

DRA Plugin Interfaces：

- Publish Resources
- Prepare Resources
- Unprepare Resources

Challenges in Adapting HAMi-core：

- libvgpu.so and ld.so.preload injection
- Environment Variables
- Temporary Directories for Cache and Lock

DRA Driver

NVIDIA DRA Driver Modification

- Introduced New FeatureGate
- New GPU Type Support
- Resource Publishing
  - type: hami-core
- ResourceClaim Preparation & HAMi-core Configuration
  - Dynamic Linker Libraries
  - Environment Variables
  - Temporary Directories
- ResourceClaim Cleanup

Why Automation is Needed?

DRA introduces a significantly more complex way to request devices compared to the traditional resource-based model.

Manual specification becomes impractical, especially in dynamic scenarios such as GPU sharing and fine-grained allocation.

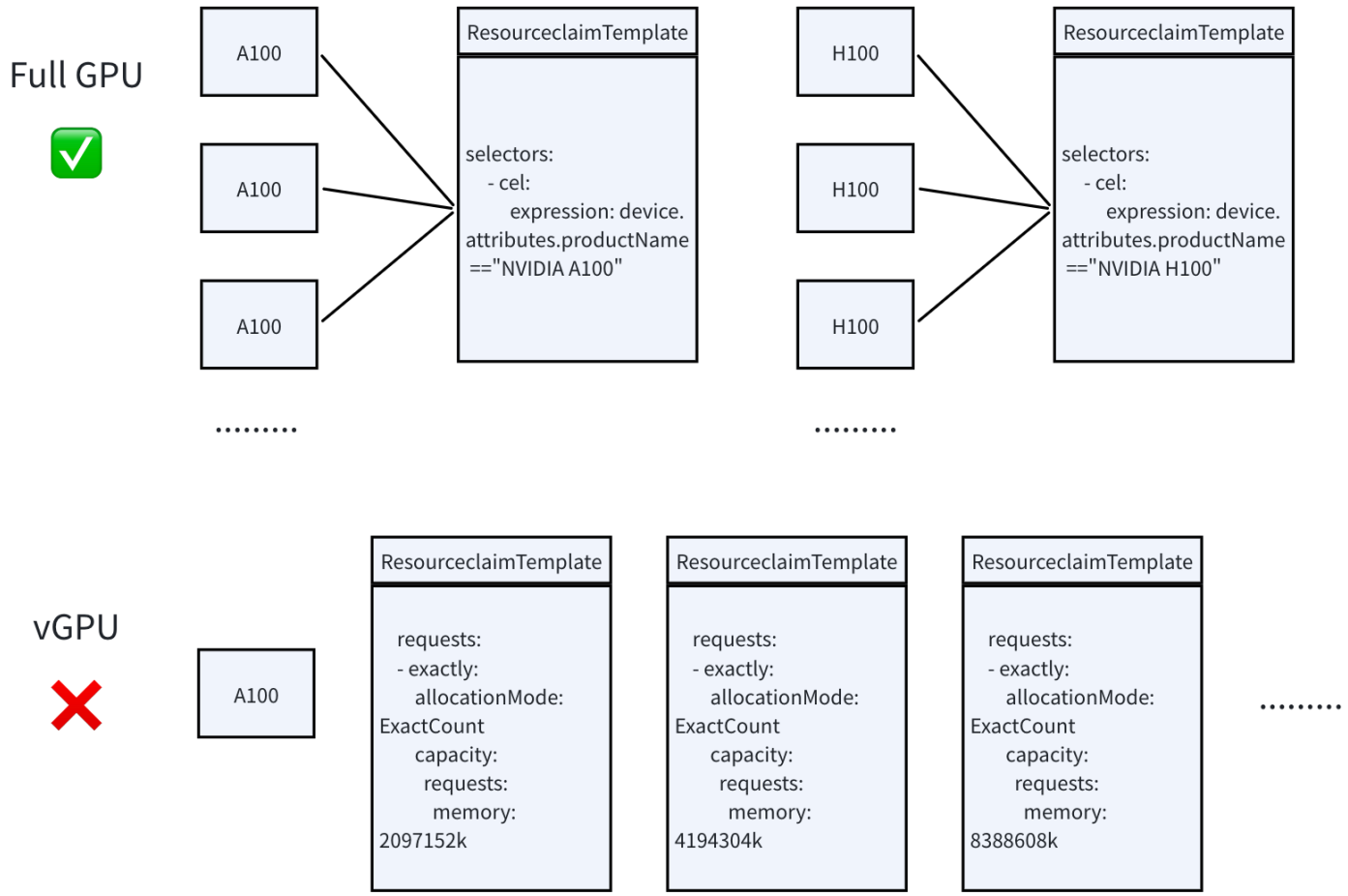
Traditional Device Plugin (Resource-based request):

```
resource:
  limit:
    nvidia.com/gpu: 1
    nvidia.com/gpucores: 30
    nvidia.com/gpumemory: 4000
```

DRA (ResourceClaim-based request):

```
spec:
  devices:
    requests:
    - exactly:
        allocationMode: ExactCount
        capacity:
          requests:
            memory: 4194304k
        count: 1
        selectors:
        - cel:
            expression: device.attributes["hami-core-
              gpu.project-hami.io"].type =="hami-gpu"
      name: gpu
```

Existing Approach 1: ResourceClaimTemplate



ResourceClaimTemplate provides limited help in simplifying DRA usage, but its applicability is restricted, especially in scenarios like flexible GPU virtualization.

- Works well for static and predictable configurations
- Lacks flexibility for dynamic, per-request customization
- Not suitable for fine-grained GPU slicing or heterogeneous devices

Existing Approach 2: DRA Extended Resource

Kubernetes provides a backward-compatible mechanism to bridge traditional resource requests with DRA.

By defining a DeviceClass with an extendedResourceName as the example,
or by using a resource name with the prefix: ***deviceclass.resource.kubernetes.io/***

```
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: gpu.example.com
spec:
  selectors:
  - cel:
      expression: device.driver == "gpu.example.com" &&
                  device.attributes["gpu.example.com"].type == "gpu"
  extendedResourceName: example.com/gpu
```

Automation is required

Limitations of Existing Approaches

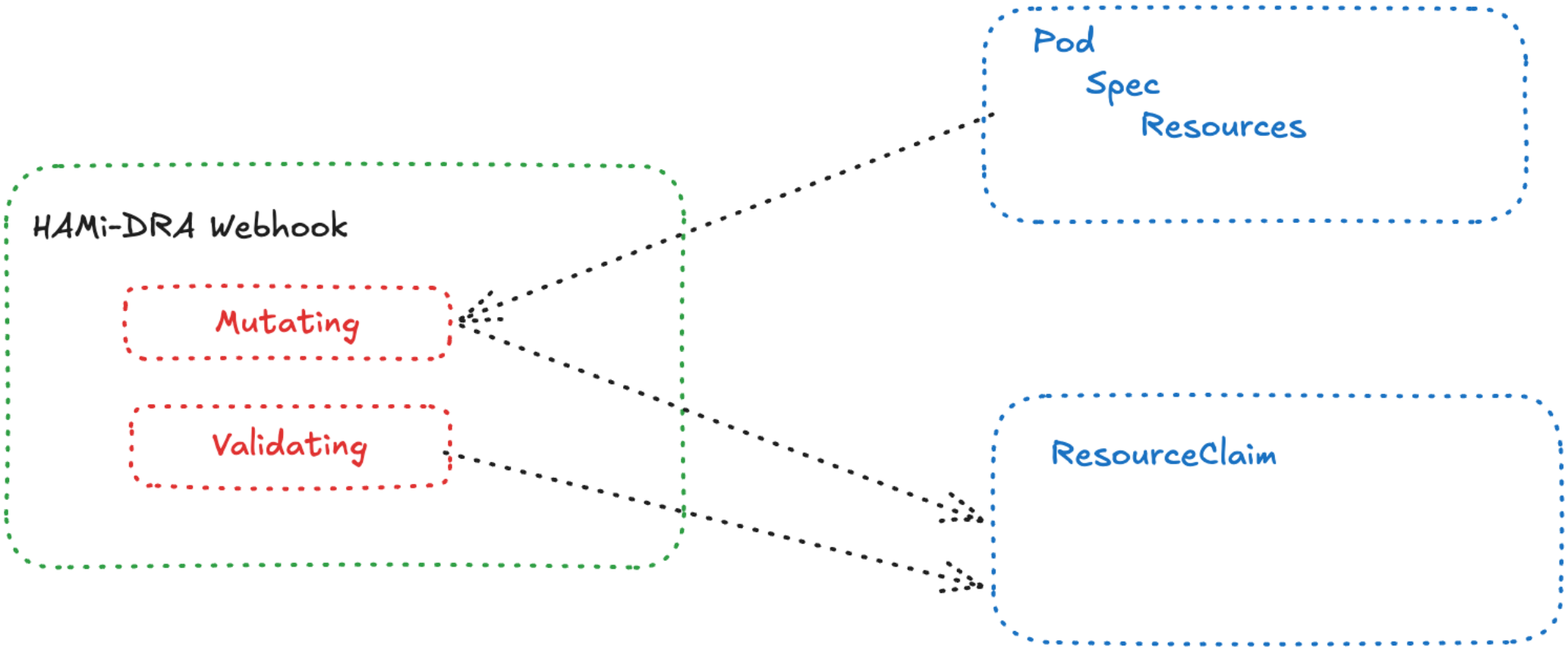
1. Lack of flexibility for dynamic workloads

2. Limited expressiveness for complex device requirements

3. Still exposes DRA complexity to end users

This leads us to introduce an automated approach based on a webhook.

HAMi DRA Webhook



Automating ResourceClaim Generation via Webhook

The webhook translates simple, user-friendly GPU requests into structured DRA ResourceClaims automatically.

Input

Users continue using familiar resource requests:

nvidia.com/gpu
nvidia.com/gpumem
nvidia.com/gpucores

Optional constraints via annotations:

nvidia.com/use-gpuuuid
nvidia.com/use-gputype

Transformation

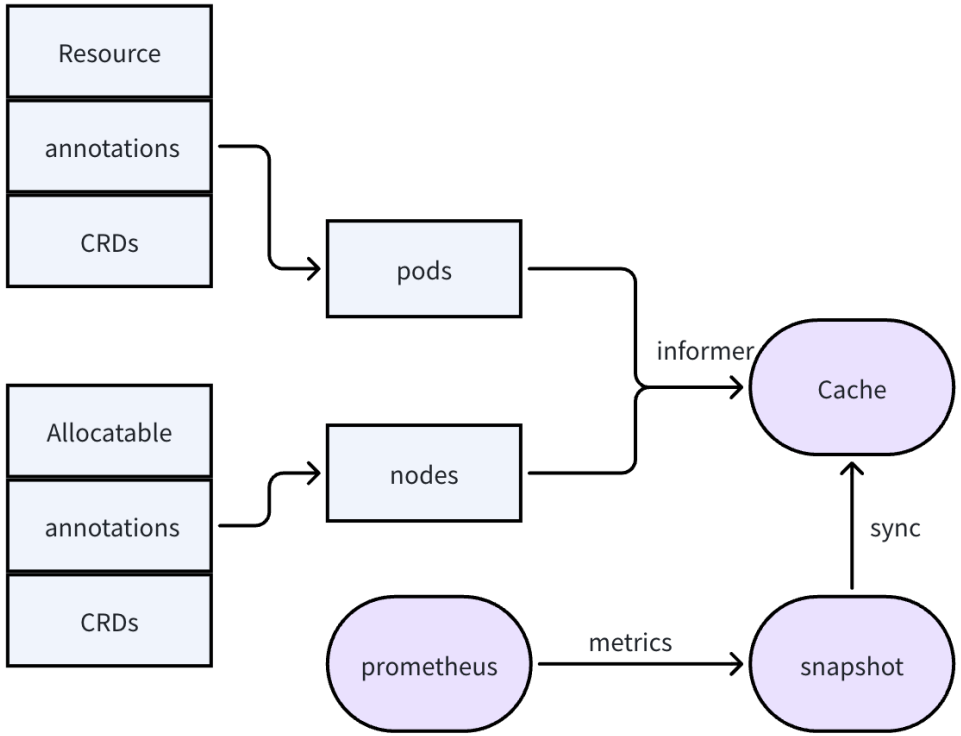
- CEL-based selectors
- Structured capacity requirements
- Precise device filtering rules

Output

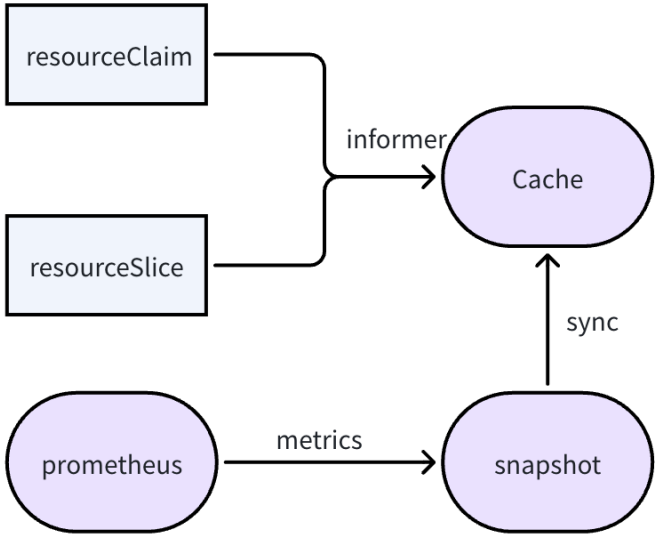
requests.exactly.capacity.requests.memory
requests.exactly.capacity.requests.cores

device.attributes["gpu.hami.io"].uuid in ["gpu-123"]
device.attributes["gpu.hami.io"].productName in ["A100"]
.....

Observability in DRA



Traditionnal scheduler monitor



Dra monitor

More efficent for developers

Before

- Device information is derived from Node.allocatable
- Resource usage is inferred from Pod resource requests
- Observability is implicitly coupled with scheduling objects (Node/Pod)
- Requires aggregation and reconstruction to build a full resource view

After (DRA Model)

•Device inventory comes from ResourceSlice

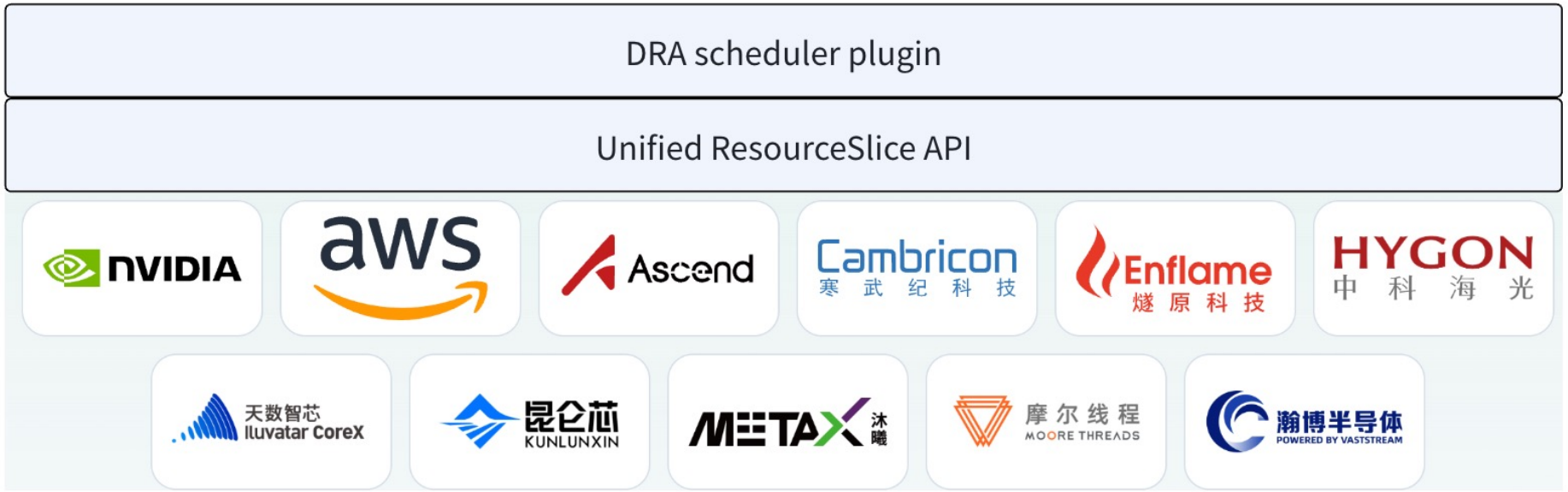
•Resource allocation is tracked via ResourceClaim

•Observability becomes resource-centric and explicit

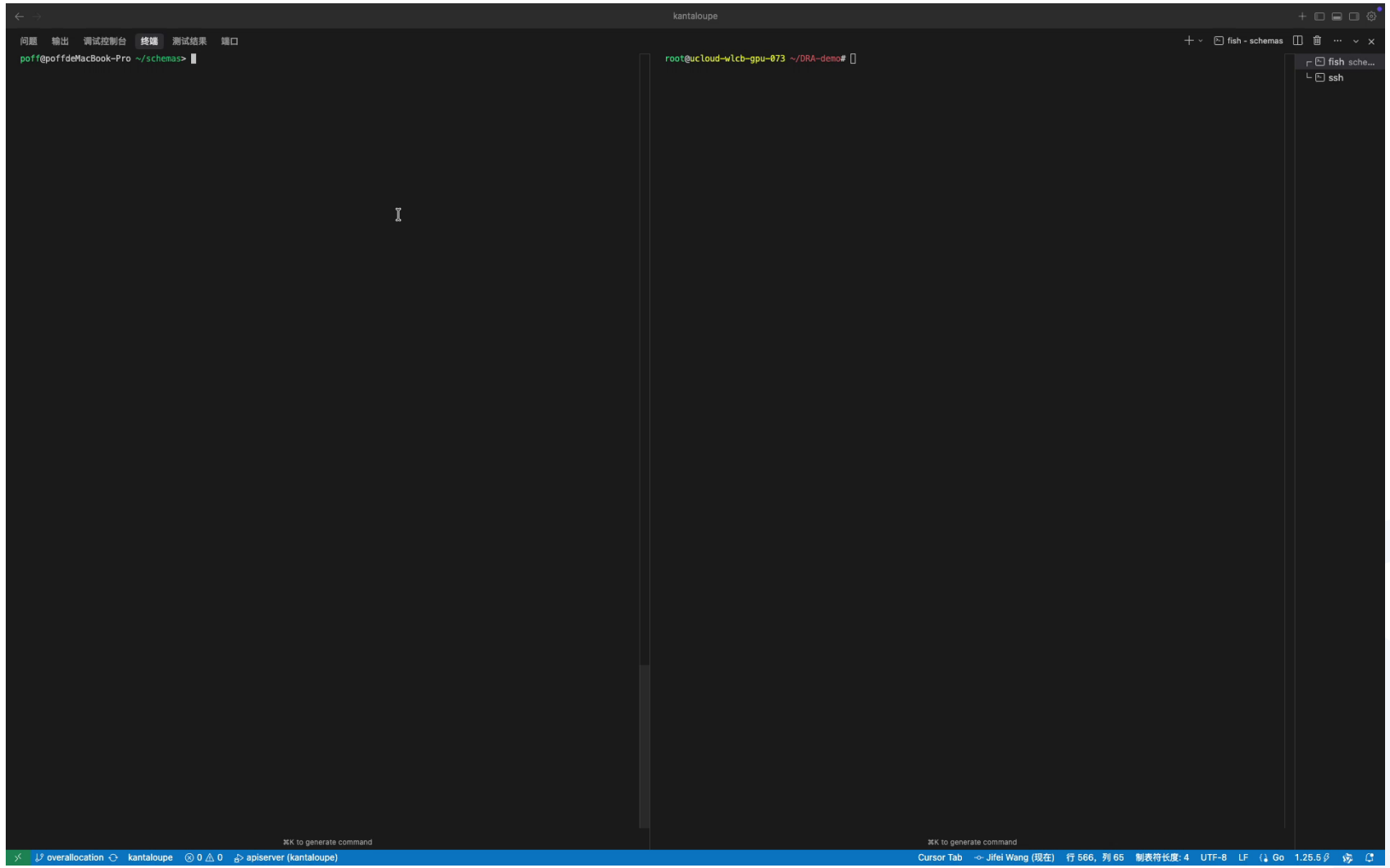
Unified Observability for Heterogeneous Devices

Core Idea

If device attributes are standardized across DRA drivers,
we can build a vendor-agnostic observability and scheduling model.



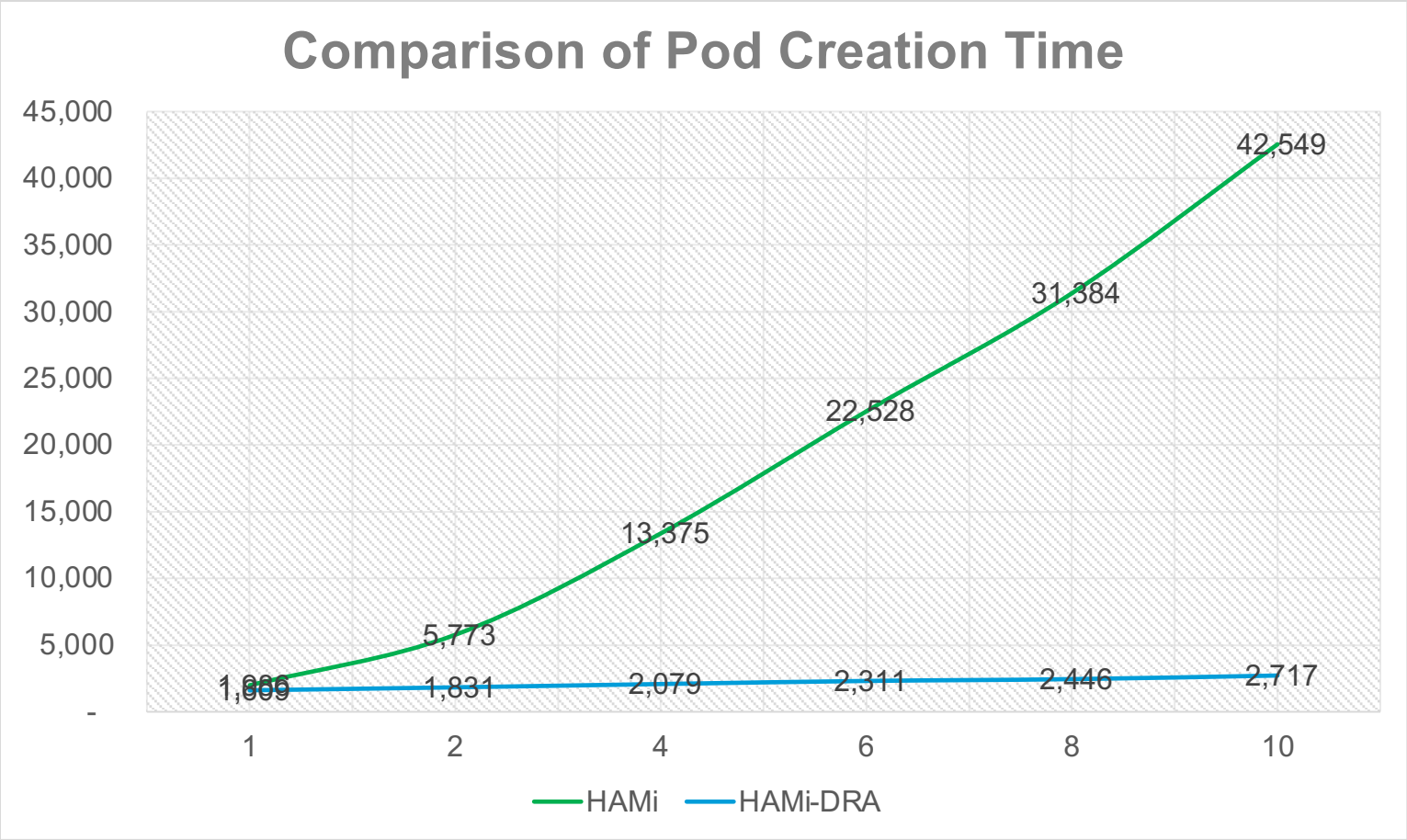
vLLM demo


kantaloupe
问题 输出 调试控制台 终端 测试结果 端口
poff@poffdeMacBook-Pro ~/schemas>
fish - schemas
root@ucloud-wlcb-gpu-073 ~/DRA-demo#
fish sche...
ssh
⌘K to generate command
⌘K to generate command
overallocation kantaloupe 0 0 apiserver (kantaloupe)
Cursor Tab Jifei Wang (现在) 行 566，列 65 制表符长度: 4 UTF-8 LF Go 1.25.5

Part 03

DRA vs Device Plugin

Benchmark


Comparison of Pod Creation Time
45,000
40,000
35,000
30,000
25,000
20,000
15,000
10,000
5,000
-
1
2
4
6
8
10
1,809
5,773
13,375
22,528
31,384
42,549
1,831
2,079
2,311
2,446
2,717
HAMi
HAMi-DRA

Part 04

Roadmap & Future Work

Roadmap & Future Work

- Device Configuration
  - Align with the features provided by the ConfigMap of HAMi NVIDIA Device Plugin
  - Support configuration of additional HAMI-Core related environment variables
- Monitoring
- Intra-node Topology Awareness
  - NUMA
  - NVLink

Roadmap & Future Work

- Standardization of Heterogeneous Device Attributes
  - resource.kubernetes.io/pcieRoot
  - resource.kubernetes.io/pciBusID
  - resource.project-hami.io/xxx ?

dra-driver-cpu / pkg / driver / dra_hooks.go

Code Blame 478 lines (412 loc) · 17 KB

```go
140  func (cp *CPUDriver) createCPUDeviceSlices() [][]resourceapi.Device {
197              cpuDevice := resourceapi.Device{
198                  Name: deviceName,
199                  Attributes: map[resourceapi.QualifiedName]resourceapi.DeviceAttribute{
200                      "dra.cpu/numaNodeID": {IntValue: &numaNode},
201                      "dra.cpu/cacheL3ID":  {IntValue: &cacheL3ID},
202                      "dra.cpu/coreType":   {StringValue: &coreType},
203                      "dra.cpu/socketID":   {IntValue: &socketID},
204                      "dra.cpu/coreID":     {IntValue: &coreID},
205                      "dra.cpu/cpuID":      {IntValue: &cpuID},
206                      // TODO(pravk03): Remove. Hack to align with NIC (DRANet). We need some standard attribute to align other resources with CPU.
207                      "dra.net/numaNode": {IntValue: &numaNode},
208                  },
209                  Capacity: make(map[resourceapi.QualifiedName]resourceapi.DeviceCapacity),
210              }
211              devicesByNuma[numaNode] = append(devicesByNuma[numaNode], cpuDevice)
212          }
213      }
```

KCD BEIJING

vLLM

Thanks.

Welcome to HAMi community

GitHub: https://github.com/Project-HAMi/HAMi

Website: https://project-hami.io/